Carleton University

Department of Systems and Computer Engineering

SYSC 4001 Operating Systems

Assignment 3 – Part I

Scheduler Simulator

Analysis - Part 1

[Bhagya Patel] - [101324150]

[Oluwatobi Olowookere] - [Student Number 2]

Professor - Gabriel Wainer

December 1st, 2025

1 Introduction and Methodology

This report analyzes three CPU scheduling algorithms for SYSC 4001 Assignment 3. The simulation models a system with 100 MB memory in six fixed partitions (40, 25, 15, 10, 8, 2 MB) running the following schedulers: **(1) External Priorities (EP)** - non-preemptive priority scheduling, **(2) Round Robin (RR)** - preemptive with 100ms quantum, and **(3) EP with RR (EP_RR)** - preemptive priority with 100ms quantum. Over 20 simulation scenarios were executed evaluating *Throughput*, *Average Turnaround Time*, *Average Wait Time*, and *Average Response Time* across CPU-bound, I/O-bound, and mixed workloads.

2 Results Analysis

2.1 External Priorities (EP) Scheduler

EP assigns CPU based on priority (lower PID = higher priority) without preemption. From testcase11, Process 1 runs 0-100ms, Process 5 runs 100-200ms, Process 10 runs 200-300ms, and Process 15 runs 300-400ms, showing strict priority ordering. Average turnaround was 220ms with average wait time of 120ms. **Strengths**: Predictable execution, low overhead, good for critical tasks. **Weaknesses**: Starvation risk for low-priority processes, poor response time for low-priority, convoy effect when short processes follow long ones.

2.2 Round Robin (RR) Scheduler

RR uses 100ms time quantum cycling through ready processes fairly. In testcase1_RR with three 50ms processes, all completed in first quantum with no preemption. In testcase3_RR with three 250ms processes, multiple rounds occurred with P1 completing at 650ms, P2 at 700ms, P3 at 750ms, yielding 700ms average turnaround and throughput of 0.004 processes/ms. **Strengths**: Fairness, excellent response time, no starvation, ideal for time-sharing. **Weaknesses**: High context switch overhead, quantum sensitivity requiring tuning.

2.3 External Priorities with Round Robin (EP_RR)

EP_RR combines priority scheduling with quantum preemption. For testcase11, results matched pure EP since no process exceeded 100ms. In testcase10, P10 (200ms) started at time 0, P5 (100ms) preempted at 50ms and completed at 150ms, then P10 resumed with quantum preemption at 250ms, completing at 300ms. Average turnaround was 200ms with P5 experiencing zero wait time. **Strengths**: Responsive to high-priority arrivals, prevents starvation via quantum limits, balances urgency and fairness. **Weaknesses**: Increased complexity, still heavily favors high-priority processes, more context switches than EP.

3 Comparative Analysis

3.1 Performance by Workload Type

CPU-Bound Processes: EP achieves highest throughput with minimal context switching but worst turnaround for low-priority processes. RR provides fair completion times with higher overhead. EP_RR behavior depends on process length vs. quantum; processes <100ms behave like EP, while processes >100ms gain fairness through preemption.

I/O-Bound Processes: From testcase4_EP_RR (I/O every 2ms for 3ms), processes naturally yield CPU during I/O enabling efficient multiplexing. All three algorithms perform similarly since voluntary CPU releases reduce scheduling policy impact.

Mixed Workloads: In testcase8_EP_RR with two 300ms CPU-bound processes and one 100ms arriving at 150ms, EP_RR demonstrated balanced priority/fairness: P5 ran 0-150ms, P1 preempted

at 150ms and completed at 250ms, P5 resumed 250-350ms, and P10 executed 400-700ms with quantum preemptions.

3.2 Key Metrics Comparison

Metric	EP	RR	EP_RR
Throughput	High	Medium	Medium-High
Turnaround Variance	Very High	Low	Medium
Response Time	Priority-dependent	Excellent	Priority-dependent
Starvation Risk	High	None	Low
Context Switch Overhead	Low	High	Medium

Throughput: EP excels with short high-priority processes but suffers when long high-priority processes block others. RR maintains consistent medium throughput except with many long processes increasing context switch overhead. EP_RR achieves medium-high throughput optimal for mixed workloads with priority separation.

Turnaround Time: EP best for high-priority short processes, worst for low-priority (high variance). RR best when all processes have equal priority, worst when quantum mismatches burst times (low variance ensures predictability). EP_RR favors high-priority like EP but quantum bounds reduce low-priority suffering (medium variance).

Response Time: RR excellent - all processes execute within one cycle (testcase3_RR all started at t=0). EP_RR good for high-priority ($\approx$0 response), poor for low-priority waiting for higher priority completion. EP identical to EP_RR, completely priority-dependent with indefinite postponement risk.

4 Memory Management and Key Findings

All schedulers use fixed-partition allocation with best-fit strategy causing inevitable internal fragmentation. In our tests, memory was never a bottleneck since all processes were 1-4 MB and total 100MB far exceeded demand. Improvements could include variable partitions, paging/segmentation, or compaction strategies.

Critical Findings: (1) *Quantum size matters* - our 100ms quantum suited 50-300ms processes; smaller quantum increases fairness but overhead. (2) *Priority creates inequality* - EP and EP_RR showed large turnaround variance based on priority. (3) *I/O provides natural preemption* - all algorithms performed similarly with I/O-bound processes. (4) *Context switch overhead is measurable* - RR showed lower CPU-bound throughput (testcase3_RR). (5) *No universal best* - each scheduler excels in different scenarios.

5 Conclusions and Recommendations

Algorithm Selection: Use *EP* for systems with critical tasks, short high-priority processes, acceptable low-priority starvation, and minimal overhead requirements (embedded systems, real-time controls). Use *RR* when fairness is paramount, in interactive/time-sharing environments, when all processes have similar importance, and response time is critical (desktop OS, time-sharing systems). Use *EP_RR* needing priority differentiation with fairness guarantees, mixed workloads with critical and background tasks, starvation prevention, and balanced throughput/response (modern servers, Linux CFS with priorities).

Our simulation of 20+ scenarios demonstrated that scheduling policy must match workload characteristics. EP maximizes throughput for priority-stratified workloads but risks starvation. RR

ensures fairness and response time at the cost of context switch overhead. EP_RR provides optimal balance for heterogeneous workloads requiring both urgency and fairness. Future work should explore aging mechanisms, dynamic priorities, multi-level feedback queues, varying quantum sizes, memory contention, multi-core simulation, and realistic I/O queuing.